

P1269R0

Three Years with the Networking TS

Published Proposal, 2018-10-07

This version:

wg21.link/P1269

Author:

[Jason Carey](#) (MongoDB)

Audience:

EWG, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest Version:

[Click here](#)

Source:

[No real reason to click here](#)

Abstract

Before the Networking TS reaches inclusion in the standard, we feel its important to share our experience using it. This paper intends to highlight limitations we've found in ergonomics and performance, along with suggestions on how they might be remedied. In particular, through tighter integration with coroutines, futures and executors.

Table of Contents

- 1 Introduction**
- 2 A summary of how we use ASIO and modifications we've made**
- 3 Areas of Concern**

- 3.1 Synchronous IO
- 3.2 Lifetime
- 3.3 Timers
- 3.4 Execution
 - 3.4.1 Relationship with the Executors TS
 - 3.4.2 Over Generalization
- 4 Key Takeaways**
 - 4.1 Core of the critique
 - 4.2 Suggestions

§ 1. Introduction

Over the past three years we've re-hosted the vast majority of the networking components in our system on top of a close proxy for the Networking TS (via its upstream in [ASIO](#)). It's provided us with opportunities to use it in a wide variety of capacities and to come to terms with its strengths, and its limitations. We believe those limitations to be compelling enough to warrant significant changes before inclusion in the standard.

§ 2. A summary of how we use ASIO and modifications we've made

At MongoDB we have an architecture that is a mixture of sync and async. For certain stages of ingress processing and for scatter/gather IO, we're in a good position to be fully async. For other portions of our operation lifecycle we are forced into a sync model, either for practical (accessing disk) or performance (the overhead of repeated context switches when needing to perform variably expensive computation) reasons.

We do all of our networking on top of ASIO, but have added to it to support:

- Handling sockets outside their `io_context` to do limited amounts of async networking synchronously
- TLS, including native support for SChannel and SecureTransport (not available in ASIO)
- A MongoDB native implementation of a Promise/Future type, via ASIO's future's extension API

§ 3. Areas of Concern

This section enumerates areas of weakness we've found in the current API.

§ 3.1. Synchronous IO

The Networking TS offers limited support for synchronous networking. In particular, it offers timeouts exclusively via its async api. A sync api caller may not early return from send/rcv/poll, and worse, risks near-perpetual hangs in the case of networking black holes. Further, its current embodiment in ASIO makes it impossible to work around yourself, as retry loops are hidden behind most every function. While these loops are required for interrupt handling on POSIX systems, it's unfortunate that core functions like `::read()` and `::write()` aren't instead named `::read_all()` and `::write_all()` as access to a lower level api would allow customization at a layer above the os native handles.

An additional quality of implementation complaint is that we've seen technical limitations around transitioning sockets between asynchronous and synchronous. For the epoll reactor, async accept followed by sync operations, forced spurious wakeups on the original context for the lifetime of those sockets. While a workaround was available, in the form of accepting sockets onto an alternate context, it's emblematic of the complexity that ASIO is attempting to hide, and the difficulty it has in doing so durably. Contexts wrap things like IO completion ports and epoll fds, and it's important to understand precisely how ASIO is using them.

§ 3.2. Lifetime

This is an artifact of how difficult callbacks are to use in C++, but it's quite awkward to correctly manage the lifetime of objects used asynchronously inside of ASIO. Objects generally need to be held by `shared_ptr` and `shared_ptr` anchors need to be added to successive callback chains. While this is easier to handle with `Future`'s, and more or less obviated with coroutines, we're currently considering a Networking TS blocked behind neither of those. I have strong reservations about the usability of any async framework in C++ built only on top of callbacks.

§ 3.3. Timers

More directly relevant for ASIO, the intermixing of lifetime for socket operations and timers to time them out is difficult to manage. Timers and their sockets need to live as least as long as both sets of callbacks, and timers need to be cancelled after socket operations have proceeded past certain points. This interplay requires careful orchestration with external concurrency primitives due to guarantees

around when timers fire (timer callbacks, after cancellation, may fire any time after cancellation, on destruction or not at all). Our experience suggests that directly attaching timeouts to async networking operations would substantially improve ease of use.

§ 3.4. Execution

§ 3.4.1. Relationship with the Executors TS

Long before the current Executors TS, ASIO had `io_service`'s with executor style APIs. Today, it has context objects with methods like `poll`, `poll_one`, `run`, `run_one`, `run_for` and `run_one_for`, which control the dedication of caller resources to processing ASIO requests. On the other side, we see methods like `post`, `dispatch` and `defer` for allowing "normal", inline or lazy execution semantics. Beyond quibbling about the names of those methods, there's a longer term problem where the Executors TS is almost certain to codify behavior and API inconsistent with the Networking Executor.

§ 3.4.2. Over Generalization

While there is a common subset of functionality that all operating systems make available, there are profound differences in how they work. If an application isn't performance sensitive, often these differences don't matter. But if an application isn't performance sensitive, it often doesn't make sense to use async networking at all, due to the added complexity callbacks introduce. And this is an area where ASIO's "one size fits all" executor strategy falls down. Your choice of `/dev/poll`, `epoll`, `kqueue`, `select` or `IOCP` is chosen for you based on your target OS, and that's all you can get. The option to use `poll` on a smaller number of sockets for a smaller number of calls; the ability to use one shot mode in `epoll`; the ability to get meaningful diagnostics about how many events are ready at once; all are out.

ASIO appears targeted for a use case that binds one `io_context` to one thread and stays on that thread. Additional cores are applied by replicating that stack across multiple threads. To the extent to which your application directly maps to that idiom, the abstraction stays tight. If you need something else (more threads, either to mix in extra computation, or to perform disk IO), the lack of control over underlying execution begins to show.

§ 4. Key Takeaways

§ 4.1. Core of the critique

As a general purpose networking layer, ASIO fails insofar as async programming in C++ fails. Without the addition of futures or coroutines, callbacks compose poorly and timeouts in particular are difficult to manage. If the sync api was more fully featured, it might be possible to ignore these problems (by avoiding them entirely), but the lack of timeout support makes the sync api unsuitable. The lack of TLS support further limits its available audience.

As a high performance networking layer, ASIO is too abstract. It fails to deliver high performance outside a specific kind of application architecture, and fails to offer direct access to OS primitives that would allow a user to work around those problems. In particular, the lack of control over underlying execution and overhead in non-share-nothing multi-threading is a real problem.

§ 4.2. Suggestions

We believe the Networking TS can be the future networking stack for the language, but it requires real and substantial modification. At a minimum, a re-work of timers to make their management easier in async code, and possible at all in sync, is needed. Putting a wait on the Networking TS until after futures or coroutines have settled more fully will solve real problems in ergonomics around async code. And waiting until after Executors have landed will allow a re-working of the internal executor inside of the TS, with better tools for both executing Networking tasks as well as waiting on their completion.